

Lab Assessment Rubrics of Digital System Design (CPE-344)

Lab Number	06										
Lab Title	Implementation of different encoder using conditional operators on FPGA Spartan 6.										
Submitted by											
Names	Registration Number										
Sumaira Safeer	FA22-BCE-019										
Rubrics name & number								Marks			
								In-Lab	Post-Lab		
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.										
	R3: Interpretation of Subject Knowledge: Ability to interpret and explain mathematical and/or visual forms, including equations, diagrams, graphics, figures, and tables.										
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.										
Design	R7: Implementing Design Strategy: Ability to execute a solution taking into consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram]										
	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.										
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.										
	R11: Tools Evaluation: Ability to simulate the experiment and them using hardware tools to verify The results.										
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.										
	R13: Management of Teamwork: Ability to appreciate, understand and work with multidisciplinary team members.										
Rubrics to follow											
Rubrics	R2	R3	R5	R6	R7	R8	R9	R11	R12	R13	
In-Lab											
Post-Lab											

Lab: 06

1. Objectives:

The objective of this lab is to get familiar with Implementation of

- 2 x 1 Line Encoder
- 4 x 2 Line Encoder
- 1 to 2 Line Decoder
- 2 to 4 Line Decoder

In Lab Tasks:

Task#01:

1. Encoder 2x1 Line

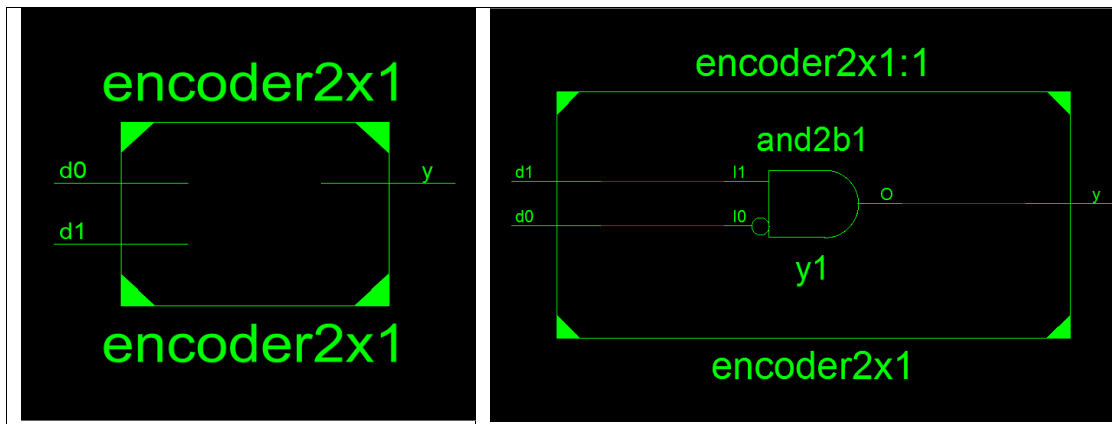
- Verilog Code

```
module encoder2x1(y, d1, d0);  
input d1, d0;  
output y;  
wire do_not;  
assign do_not = ~d0;  
assign y = do_not && d1;  
endmodule
```

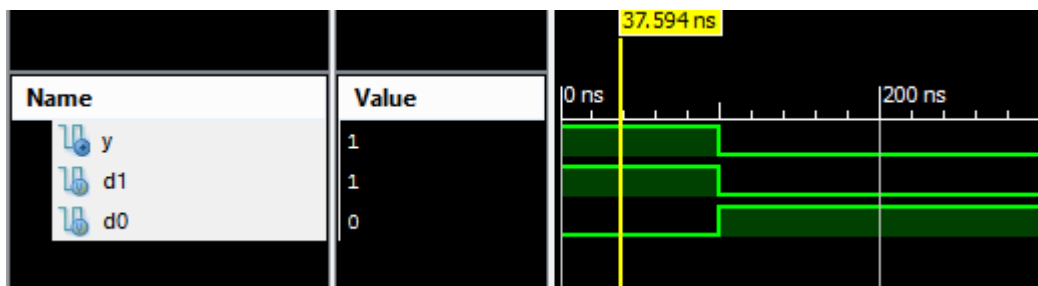
- Test Bench

```
module encoder2x1TestBench;  
    reg d1;  
    reg d0;  
    wire y;  
    encoder2x1 uut (  
        .y(y),  
        .d1(d1),  
        .d0(d0)  
    );  
    initial begin  
        d1 = 1; d0 = 0;  
        #100;  
        d1 = 0; d0 = 1;  
    end  
endmodule
```

- RTL Schematic



- **Test Bench**



- **Truth Table**

INPUT		OUTPUT
D ₀	D ₁	X
1	0	0
0	1	1

2. Encoder 4x2 Line

- **Verilog Code**

```

module encoder4x2(a1, a0, y3, y2, y1, y0);
input y3, y2, y1, y0;
output a1, a0;
assign a0 = y1 || y3;
assign a1 = y2 || y3;
endmodule

```

- **Test Bench**

```

module encoder4x2Test;
reg y3, y2, y1, y0;
wire a1, a0;
encoder4x2 uut (
.a1(a1),

```

```

        .a0(a0),
        .y3(y3),
        .y2(y2),
        .y1(y1),
        .y0(y0)
    );
    initial begin
        y3 = 0; y2 = 0;
        y1 = 0; y0 = 1;
        #100;

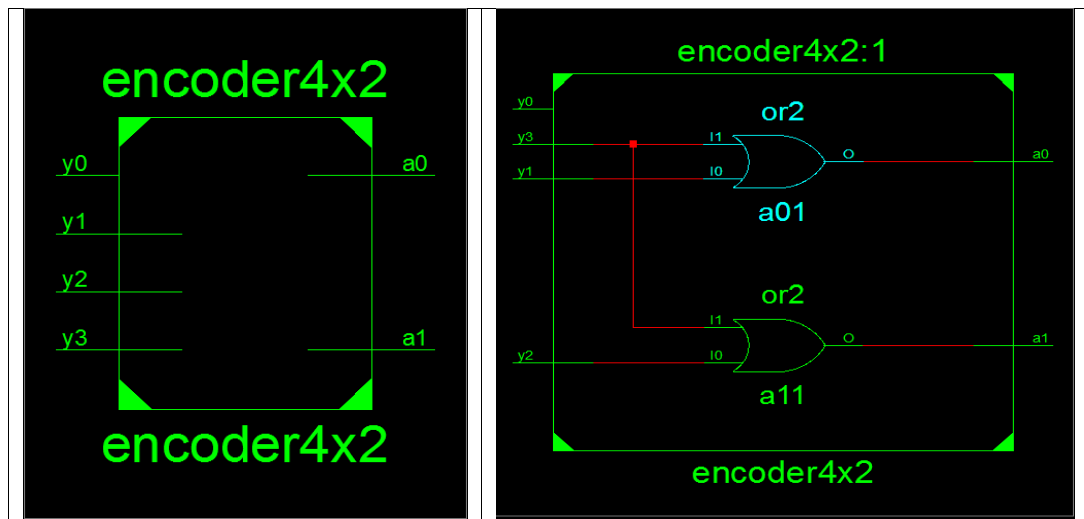
        y3 = 0; y2 = 0;
        y1 = 1; y0 = 0;
        #100;

        y3 = 0; y2 = 1;
        y1 = 0; y0 = 0;
        #100;

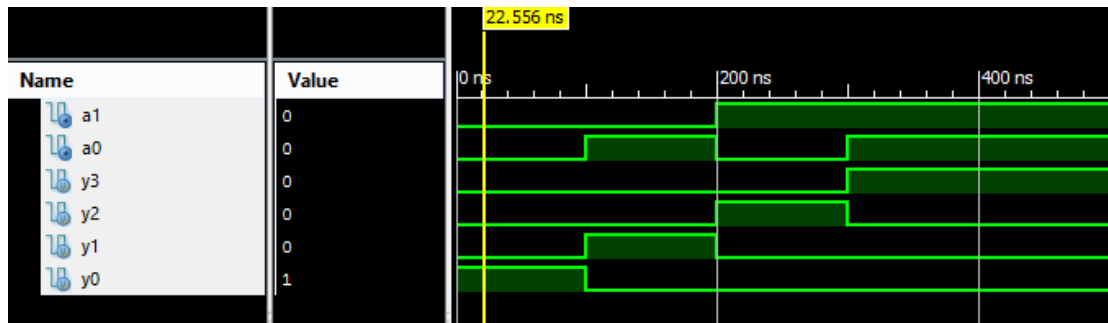
        y3 = 1; y2 = 0;
        y1 = 0; y0 = 0;
        #100;
    end
endmodule

```

- RTL Schematic



- Test Bench



- **Truth Table**

INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

3. Encoder 8x3 Line

- **Verilog Code**

```

module encoder8x3(a2, a1, a0, d7, d6, d5, d4, d3, d2, d1, d0);
input d7, d6, d5, d4, d3, d2, d1, d0;
output a2, a1, a0;
assign a2 = d4 || d5 || d6 || d7;
assign a1 = d2 || d3 || d6 || d7;
assign a0 = d1 || d3 || d5 || d7;
endmodule

```

- **Test Bench**

```

module encoder8x3TestBench;
reg d7, d6, d5, d4, d3, d2, d1, d0;
wire a2, a1, a0;
encoder8x3 uut (
    .a2(a2), .a1(a1),
    .a0(a0), .d7(d7),
    .d6(d6), .d5(d5),
    .d4(d4), .d3(d3),
    .d2(d2), .d1(d1),
    .d0(d0));
initial begin
    d7 = 0; d6 = 0;

```

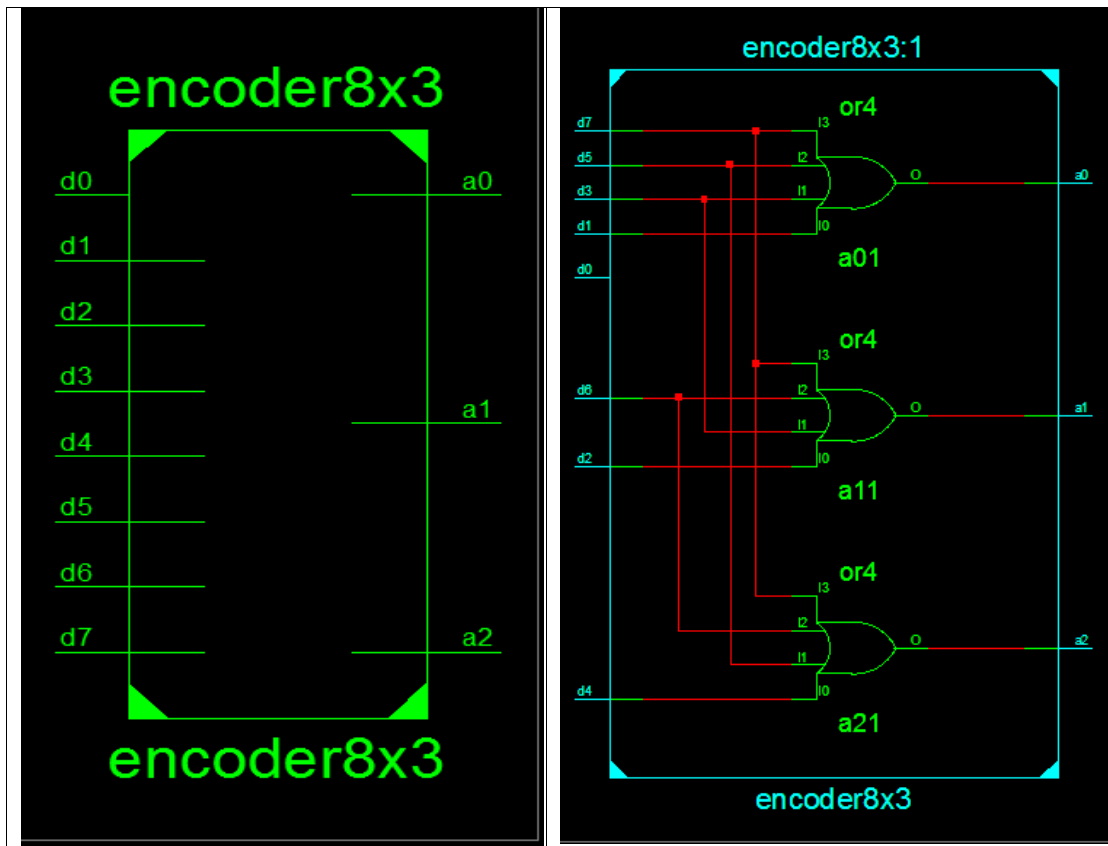
```

d5 = 0; d4 = 0;
d3 = 0; d2 = 0;      d1 = 0; d0 = 1;
#100;
d7 = 0; d6 = 0;
d5 = 0; d4 = 0;
d3 = 0; d2 = 0;
d1 = 2; d0 = 0;
#100;
d7 = 0; d6 = 0;
d5 = 0; d4 = 0;
d3 = 0; d2 = 1;
d1 = 0; d0 = 0;
#100;
d7 = 0; d6 = 0;
d5 = 0; d4 = 0;
d3 = 1; d2 = 0;
d1 = 0; d0 = 0;
#100;
d7 = 0; d6 = 0;
d5 = 0; d4 = 1;
d3 = 0; d2 = 0;
d1 = 0; d0 = 0;
#100;
d7 = 0; d6 = 0;
d5 = 1; d4 = 0;
d3 = 0; d2 = 0;
d1 = 0; d0 = 0;
#100;
d7 = 0; d6 = 1;
d5 = 0; d4 = 0;
d3 = 0; d2 = 0;
d1 = 0; d0 = 0;
#100;
d7 = 1; d6 = 0;
d5 = 0; d4 = 0;
d3 = 0; d2 = 0;
d1 = 0; d0 = 0;

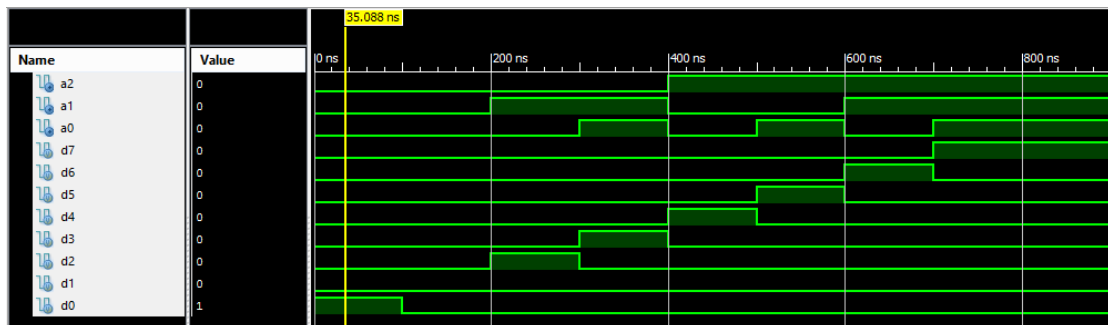
end
endmodule

```

- RTL Schematic



- Test Bench



- Truth Table

INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

4. Decoder 1 to 2 Line

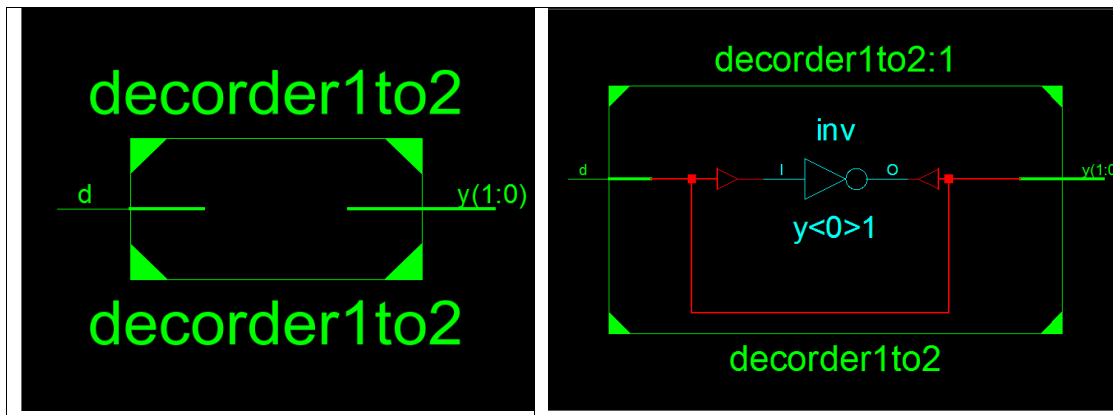
- Verilog Code

```
module decoder1to2(y, d);
input d;
output [1:0]y;
assign y[0] = ~d;
assign y[1] = d;
endmodule
```

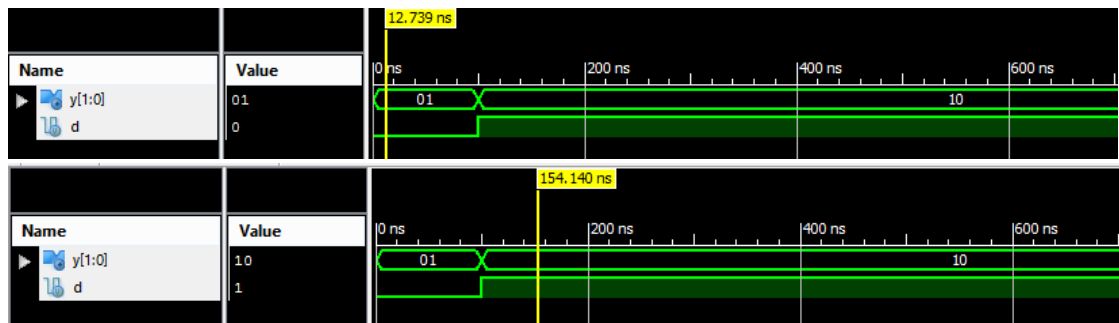
- Test Bench

```
module decoder1to2Test;
reg d;
wire [1:0] y;
decoder1to2 uut (
.y(y), .d(d));
initial begin
d = 0;
#100;
d = 1;
#100;
end
endmodule
```

- RTL Schematic



- Waveform



- Truth Table

Input	Output	
A	D ₀	D ₁
0	1	0
1	0	1

5. Decoder 2 to 4 Line

- Verilog Code

```

module decoder2to4(a, d);
input [1:0]d;
output [3:0]a;
assign a[0] = d[1] && d[0];
assign a[1] = d[1] && ~d[0];
assign a[2] = ~d[1] && d[0];
assign a[3] = ~d[1] && ~d[0];
endmodule

```

- Test Bench

```

module decoder2to4Test;
reg [1:0] d;

```

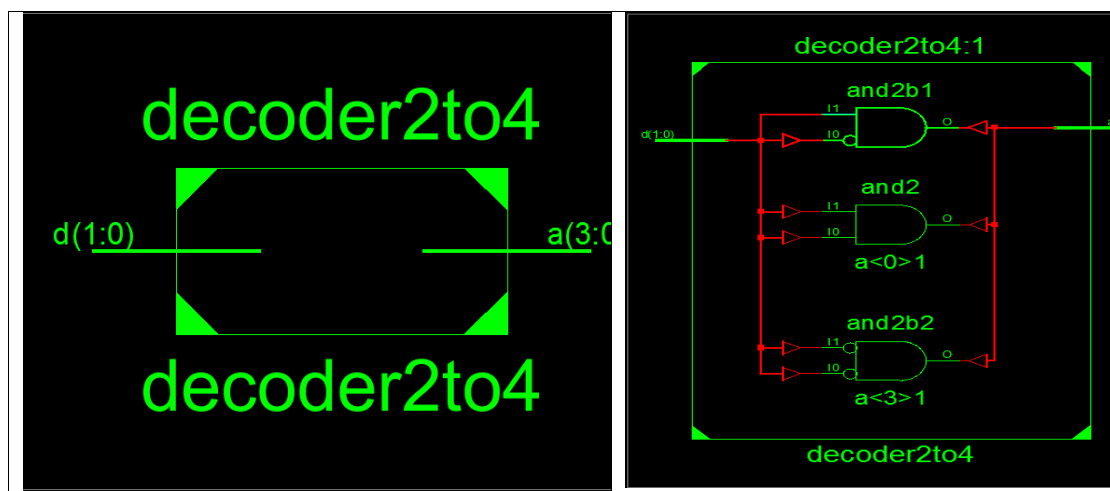
```

wire [3:0] a;
decoder2to4 uut (
    .a(a), .d(d));
initial begin
    d = 2'b00;    #100;
    d = 2'b01;    #100;
    d = 2'b10;    #100;
    d = 2'b11;    #100;

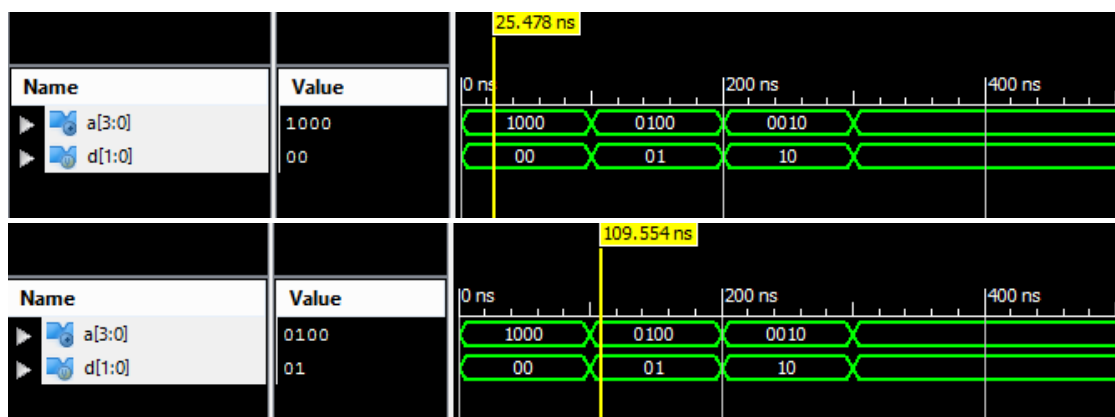
end
endmodule

```

- RTL Schematic



- Waveform



- Truth Table

A	B	Q ₀	Q ₁	Q ₂	Q ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

6. Decoder 3 to 8 Line

• Verilog Code

```

module decoder3to8(a, d);
input [2:0]d;
output [7:0]a;

assign a[0] = d[2] && d[1] && d[0];
assign a[1] = d[2] && d[1] && ~d[0];
assign a[2] = d[2] && ~d[1] && d[0];
assign a[3] = d[2] && ~d[1] && ~d[0];
assign a[4] = ~d[2] && d[1] && d[0];
assign a[5] = ~d[2] && d[1] && ~d[0];
assign a[6] = ~d[2] && ~d[1] && d[0];
assign a[7] = ~d[2] && ~d[1] && ~d[0];
endmodule

```

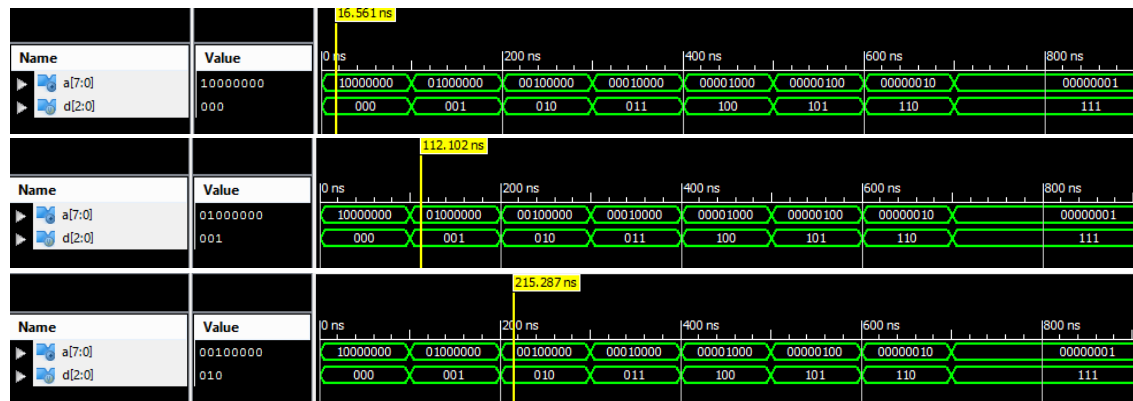
• Test Bench

```

module decoder3to8Test;
reg [2:0] d;
wire [7:0] a;
decoder3to8 uut (
    .a(a), .d(d));
initial begin
    d = 3'b000; #100;
    d = 3'b001; #100;
    d = 3'b010; #100;
    d = 3'b011;    #100;
    d = 3'b100; #100;
    d = 3'b101; #100;
    d = 3'b110;    #100;
    d = 3'b111;
end
endmodule

```

- Waveform



- Truth Table

Inputs			Outputs							
x	y	z	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1